

CONFIDENTIAL ACADEMIC SUBMISSION - CSE499 review and grading only.

Mercury Project

Requirements Document

Functional and nonfunctional requirements for Mercury RC 1.2.

Course	CSE499 Senior Project
Author	Matthew D. Barker
Version	RC 1.2 (1.2.0-rc.1)
Date	July 2026
Use	Academic review and grading only

This document and related project materials may not be publicly posted, redistributed, showcased, or shared outside the course review process without prior written permission from the copyright owner.

Document Purpose

This document defines what Mercury must do for the CSE499 senior project. It establishes scope, functional requirements, nonfunctional requirements, acceptance criteria, assumptions, constraints, and requirement traceability for Mercury RC 1.2.

Project Overview

Mercury is a reusable secure communications framework for protected data exchange between software components. It protects application payloads before transport, authenticates SecureEnvelope content, rejects replay attempts, and separates cryptographic logic from transport and encoding concerns.

Mercury is not a chat application. It is a protected data exchange layer that supports text, binary payloads, file-transfer demonstrations, telemetry-style payloads, and service-to-service communication scenarios.

Scope

In Scope

- Protect payloads before transport and release plaintext only after successful validation.
- Support AES-GCM and ChaCha20-Poly1305 authenticated-encryption providers.
- Authenticate the payload and canonical SecureEnvelope fields.
- Reject tampered, malformed, replayed, unauthorized, or wrong-key communications.
- Support interchangeable Binary and JSON envelope codecs.
- Support interchangeable in-memory, framed TCP, and FileDrop transports.
- Provide process-local replay protection with expiration and bounded storage.
- Provide cross-platform console and Windows WinForms demonstration applications.
- Provide automated xUnit tests for framework, provider, transport, codec, replay, and pipeline behavior.
- Provide formal project documentation for CSE499 evaluation.

Out of Scope

- Public key infrastructure and certificate authority integration.
- Production identity management and human identity proofing.
- Long-term key rotation, key exchange, and hardware-backed key storage.
- DoD certification, classified use, or production deployment claims.
- Full side-channel hardening or formal cryptographic proof.
- Persistent distributed replay storage.
- Production-scale denial-of-service certification, fuzz certification, or long-duration stress certification.

Stakeholders

Stakeholder	Interest
Student developer	Demonstrates senior-level architecture, implementation, testing, documentation, and defensive programming.
CSE499 evaluator	Needs clear requirements, a runnable project, visible security behavior, test evidence, and documented limitations.
Application developer	Uses IMercuryClient to send and receive protected payloads without coupling security logic to a transport.
System maintainer	Needs clear project boundaries, modular plugins, stable contracts, and safe failure behavior.

Assumptions

1. Cryptographic keys are provisioned before Mercury is used.
2. The .NET implementations of AES-GCM and ChaCha20-Poly1305 operate correctly.
3. Keys are generated with sufficient entropy and protected outside the framework.
4. The transport may be observable, unreliable, delayed, modified, or hostile.
5. The application treats unsuccessful receive results as rejected communications.
6. Replay-protection state is retained for the lifetime of the receiving context.
7. The operating system, .NET runtime, and trusted application process are not compromised.
8. Attackers do not possess the correct secret key material.

Functional Requirements

ID	Name	Requirement	Acceptance Criteria
FR-001	Send Protected Payload	Mercury shall accept a non-empty payload and valid cryptographic context, protect the payload, encode a SecureEnvelope, and send only the encoded protected frame through the configured transport.	SendAsync sends a frame that does not contain the original plaintext payload.
FR-002	Receive Protected Payload	Mercury shall receive an encoded frame, decode it, authenticate and decrypt it, apply replay protection, and return plaintext only after all checks succeed.	ReceiveAsync returns success and the original payload only for a valid protected communication.
FR-003	Payload Confidentiality	Mercury shall protect payload confidentiality using the configured authenticated-encryption provider.	Captured transport frames do not expose the original payload.
FR-004	Tamper Detection	Mercury shall reject changes to protected payload bytes or authenticated SecureEnvelope fields.	Tampered content returns AuthenticationFailed and releases no plaintext.
FR-005	Replay Rejection	Mercury shall reject a sender and replay-token combination that was already accepted within the replay window.	The first valid delivery succeeds and the duplicate is rejected as replay.
FR-006	Wrong-Key Failure	Mercury shall fail safely when incorrect key material is used.	Authentication fails and plaintext is not released.
FR-007	Crypto Provider Abstraction	Mercury Core shall depend on ICryptoProvider rather than a concrete provider.	AES-GCM and ChaCha20-Poly1305 operate through the same core pipeline.
FR-008	Transport Abstraction	Mercury Core shall depend on ITransport for complete frame delivery.	The same core client works with in-memory, TCP, and FileDrop transports.
FR-009	Codec Abstraction	Mercury shall encode and decode SecureEnvelope values through IEnvelopeCodec.	Binary and JSON codecs preserve the complete envelope structure.
FR-010	Authenticated Envelope Context	Mercury shall include canonical envelope context in provider authentication.	Changes to version, IDs, timestamp, algorithms, replay token, or metadata fail authentication.
FR-011	Metadata Support	Mercury shall support optional header and footer metadata.	Metadata survives codec round trips and is authenticated by completed providers.
FR-012	Binary Payload Support	Mercury shall support arbitrary byte payloads and chunked file demonstrations.	Binary and file payloads are recovered byte-for-byte after successful validation.
FR-013	Explicit Client Identity	Each Mercury client shall have an explicit KeyId used to validate intended recipient context.	A client rejects a protected envelope addressed to another recipient.
FR-014	Bounded Replay Storage	The in-memory replay protector shall expire entries and enforce a configurable maximum active-entry count.	Expired records are removed and new records are rejected when capacity is reached.
FR-015	Frame Size Guard Rails	In-memory and TCP transports shall reject frames larger than the configured maximum.	Oversized frames fail before unbounded allocation or transport processing.

ID	Name	Requirement	Acceptance Criteria
FR-016	Controlled Failure Results	Receive-side validation failures shall return controlled results without plaintext where possible.	Decode, authentication, replay, and provider failures do not release plaintext.
FR-017	Demonstration Applications	Mercury shall provide console and WinForms hosts that demonstrate successful exchange and security failure scenarios.	An evaluator can run normal, tamper, replay, wrong-key, transport, and file scenarios.

Nonfunctional Requirements

ID	Name	Requirement
NFR-001	Modularity	Crypto, transport, codec, replay protection, abstractions, and core coordination shall remain separate.
NFR-002	Misuse Resistance	Public APIs shall reject empty payloads, missing context values, invalid key identifiers, and unsupported provider input.
NFR-003	Testability	The design shall support isolated unit, contract, pipeline, transport, and integration tests.
NFR-004	Maintainability	Interfaces and project boundaries shall allow providers and transports to be added without rewriting Mercury Core.
NFR-005	Demo Clarity	The demos shall make provider, transport, codec, send, receive, tamper, replay, and wrong-key behavior visible.
NFR-006	Safe Failure	Invalid, malformed, replayed, unauthorized, or tampered communications shall not release plaintext.
NFR-007	Thread Safety	Shared transports, replay storage, and key lookup used concurrently shall behave predictably.
NFR-008	Resource Control	Replay storage and supported transport frame sizes shall be bounded.
NFR-009	Academic Transparency	Documentation shall identify implemented capabilities, pending work, and out-of-scope production concerns without overclaiming.
NFR-010	Cross-Platform Hosting	Framework projects and the console demo shall remain usable outside Windows where their target frameworks permit.

Constraints

- C# and .NET are the implementation platform.
- Mercury.Abstractions and Mercury.Core target .NET Standard 2.0-compatible contracts where required by the architecture.
- The WinForms demonstration targets Windows; the console demonstration provides a cross-platform host.
- The test project uses xUnit on .NET 10.
- AES-GCM and ChaCha20-Poly1305 are the completed RC 1.2 providers.
- AES-CCM remains pending and its two tests remain intentionally skipped.
- Replay protection is in-memory and process-local.
- The project must remain academically reviewable and must not claim production or government certification.

Requirement Traceability

Requirement	Design Element	Test or Demo Evidence	Status
FR-001/FR-002	IMercuryClient send and receive pipeline	MercuryPipelineTests; WinForms and console round-trip demonstrations	Implemented

Requirement	Design Element	Test or Demo Evidence	Status
FR-003/FR-004/FR-010	AES-GCM and ChaCha20-Poly1305 providers; authenticated canonical envelope data	CryptoProviderContractTests; AuthenticatedEnvelopeTamperTests; tamper demo	Implemented
FR-005/FR-014	IReplayProtector and InMemoryReplayProtector	InMemoryReplayProtectorTests; replay pipeline tests; replay demo	Implemented
FR-006	Provider authentication failure with mismatched keys	Wrong-key provider and pipeline tests; wrong-key demo	Implemented
FR-007/FR-008/FR-009	ICryptoProvider, ITransport, and IEnvelopeCodec boundaries	Provider/codecs matrix tests; in-memory, TCP, FileDrop, Binary, and JSON tests	Implemented
FR-011/FR-012	Metadata and ReadOnlyMemory payload contracts	Codec round-trip, metadata tamper, binary payload, large payload, and file scenarios	Implemented
FR-013	Explicit client KeyId validation	Recipient mismatch and pipeline failure tests	Implemented
FR-015	In-memory and TCP frame limits	Transport limit and oversized-frame tests	Implemented
FR-016	FailureReason and payload release controls	Decode, provider, transport, authentication, replay, and cancellation tests	Implemented
FR-017	Mercury.Demo.Console and Mercury.Demo.WinForms	Manual demo verification	Implemented
AES-CCM	Pending provider contribution	Two intentionally skipped tests	Pending

Acceptance Summary

Mercury RC 1.2 satisfies the defined CSE499 scope when the solution builds, the full automated suite completes with zero failures, the completed providers and codecs pass their contract and pipeline tests, and the console and WinForms demonstrations show successful exchange, tamper rejection, replay rejection, and wrong-key failure. The latest recorded verification discovered 330 tests, passed 328, skipped the two pending AES-CCM tests, and reported zero failures.